

PROFUNDIZACIÓN FORMAL, TÉCNICA Y OBJETIVA EN SEGURIDAD INFORMÁTICA

PROYECTO: SISTEMA WEB PWA DE GESTIÓN Y TOMA DE PEDIDOS - LEOFIT SOLUTIONS

UTP - CURSO INTEGRADOR II (100000S12F) | GRUPO 01

1. PROPÓSITO Y CONTEXTO DE LA RETROALIMENTACIÓN DOCENTE

Durante la evaluación académica de proyectos de software en ingeniería de sistemas, los docentes y jurados calificadores suelen observar que la sección de seguridad se redacta con afirmaciones genéricas (como "el sistema es seguro porque usamos JWT y Bcrypt").

La exigencia de "profundizar en la seguridad de manera formal y técnica pero objetiva y entendible" significa:

- 1. **Formal y Técnica:** Detallar con precisión los algoritmos criptográficos utilizados, longitud de claves, vectores de ataque analizados y código fuente de mitigación.
- 2. **Objetiva:** Enfocar la seguridad en el contexto real del negocio de LeoFit (Gamarra, pedidos multicanal por WhatsApp, cobros y catálogo textil), identificando qué activos están en riesgo.
- 3. **Entendible:** Explicar el principio de **Defensa en Profundidad (*Defense-in-Depth*)** mediante un modelo de amenazas estructurado y demostrable ante el jurado.

2. MODELO DE AMENAZAS DEL SISTEMA LEOFIT (MATRIZ STRIDE)

Para estructurar la seguridad de forma metódica, el sistema LeoFit se evaluó bajo el modelo de amenazas **STRIDE** (desarrollado por Microsoft y estándar en ingeniería de software):

Vector STRIDE	Amenaza en el Contexto de LeoFit	Riesgo de Negocio	Control Técnico Implementado en el Código
S - Spoofing (Suplantación)	Un atacante intenta hacerse pasar por Víctor Cárdenas (Admin) adivinando contraseñas por fuerza bruta.	Acceso indebido a finanzas y modificación fraudulenta de precios.	Hashing con bcrypt (10 rounds de salting) + Rate Limiting estricto (máx. 20 intentos por 15 min en /api/auth/login).
T - Tampering (Manipulación)	Un cliente intercepta la petición HTTP y altera el precio total de una prenda de S/ 89.90 a S/ 1.00.	Venta por debajo del costo y quiebre comercial.	Validación autoritativa en el Backend: El servidor ignora el total enviado por el cliente y recalcula los precios consultando la BD con validación Zod.

R - Repudiation (<i>Repudio</i>)	Un operador cambia el estado de un pedido a "Cancelado" y luego niega haberlo hecho.	Descoordinación de almacén y pérdida de pedidos.	Tabla inmutable de auditoría (audit_logs) con triggers en PostgreSQL que registran user_id, ip_address, timestamp_utc y old_value/new_value .
I - Information Disclosure (<i>Fuga de Información</i>)	Inyección SQL en el buscador de pedidos para extraer la base de datos de teléfonos y nombres de clientes.	Violación de la Ley N° 29733 de Protección de Datos Personales de Perú.	Consultas 100% parametrizadas con el patrón Repositorio en PostgreSQL (\$1, \$2). Ninguna concatenación de cadenas.
D - Denial of Service (<i>Denegación de Servicio</i>)	Bots envían miles de peticiones por segundo al endpoint de catálogo para colapsar el servidor.	Clientes y vendedores no pueden registrar pedidos en hora punta.	Middleware express-rate-limit con ventana deslizante (máximo 100 peticiones por 15 minutos por IP).
E - Elevation of Privilege (<i>Escalamiento de Privilegios</i>)	Un vendedor con rol VENDEDOR envía peticiones directas al endpoint /api/dashboard o altera el stock de almacén.	Sabotaje de inventario y fuga de métricas gerenciales de facturación.	Middleware RBAC requireRole(['ADMIN']) que valida criptográficamente el claim role dentro del JWT firmado con HMAC-SHA256.

3. ARQUITECTURA DE DEFENSA EN PROFUNDIDAD (LOS 7 NIVELES TÉCNICOS)

El sistema implementa 7 capas concéntricas de seguridad técnica:

```

[1. Capa de Red y Transporte: HTTPS / TLS 1.3 con Cifrado ECDHE]
├── [2. Capa Perimetral HTTP: Cabeceras Helmet + CORS Whitelist + Rate Limiting]
│   ├── [3. Capa de Autenticación: Bcrypt Salteado + JWT Stateless HS256]
│   │   ├── [4. Capa de Autorización: RBAC con Principio de Menor Privilegio]
│   │   │   ├── [5. Capa de Validación de Entrada: Esquemas Estrictos Zod]
│   │   │   └── [6. Capa de Persistencia: Consultas Preparadas en
│   │   │       PostgreSQL]
│   │   └── [7. Capa de Trazabilidad: Auditoría Inmutable y
│   │       Backups WAL]

```

NIVEL 1: Cifrado en Tránsito y Red (Transport Layer Security - TLS 1.3)

- **Algoritmo de Intercambio de Claves:** Diffie-Hellman Efímero de Curva Elíptica (**ECDHE - Elliptic Curve Diffie-Hellman Ephemeral**), garantizando **Perfect Forward Secrecy (PFS)**. Si la clave privada del servidor se viera comprometida en el futuro, las sesiones pasadas no pueden ser descifradas.
- **Cifrado Simétrico Autenticado (AEAD): AES-256-GCM (Galois/Counter Mode)**, proveyendo simultáneamente confidencialidad e integridad criptográfica en cada paquete transmitido.

- **Cabecera HSTS:** `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` forzando a los navegadores a conectarse únicamente mediante HTTPS.

NIVEL 2: Perímetro y Endurecimiento HTTP (Helmet & CORS)

En `backend/src/app.ts`, se configura la librería **Helmet** para inyectar cabeceras defensivas que neutralizan vectores comunes del navegador:

4. **Anti Clickjacking** (`'X-Frame-Options: DENY'`): Impide que atacantes incrusten la plataforma LeoFit dentro de un `<iframe>` invisible en un sitio malicioso para inducir clics involuntarios de compra.
5. **Anti MIME Sniffing** (`'X-Content-Type-Options: nosniff'`): Prohíbe al navegador interpretar archivos JSON o imágenes como scripts ejecutables JavaScript.
6. **Protección de Referencias** (`'Referrer-Policy: strict-origin-when-cross-origin'`): Evita que tokens o parámetros sensibles de la URL se filtren hacia servidores de analítica externa.
7. **Política CORS Estricta:** Se deniegan peticiones de orígenes cruzados arbitrarios, aceptando únicamente el dominio de producción de LeoFit y localhost en desarrollo.

NIVEL 3: Autenticación Robusta y Criptografía de Credenciales (Bcrypt + JWT)

1. Hashing de Contraseñas con Bcrypt

- **Algoritmo:** Función de derivación de claves basada en el cifrador por bloques *Blowfish*.
- **Factor de Costo (Work Factor / Rounds):** **10 rondas** ($2^{10} = 1,024$ iteraciones).
- **Salting Criptográfico:** Se genera una sal aleatoria de **128 bits** por cada contraseña, lo que garantiza que dos usuarios con la misma contraseña tengan hashes completamente diferentes en la base de datos, anulando por completo los ataques de tablas arcoíris (*Rainbow Tables*).
- **Resistencia a Ataques por GPU:** A diferencia de MD5 o SHA-256 (que son funciones de hash criptográfico rápidas diseñadas para firmas de datos y fácilmente vulnerables a fuerza bruta masiva por tarjetas gráficas), Bcrypt es intensivo en memoria (*memory-hard*), limitando la velocidad de cracking a pocos intentos por segundo por hardware.

2. Emisión y Verificación de Tokens JWT (RFC 7519)

- **Algoritmo de Firma:** **HMAC-SHA256** (`'HS256'`), combinando el secreto del servidor con el payload mediante la función hash criptográfica SHA-256.
- **Estructura del Payload:**

```
{
  "sub": "usr_98a72b",
  "email": "admin@leofit.pe",
  "role": "ADMIN",
  "iat": 1727280000,
  "exp": 1727366400
}
```

- **Tiempo de Expiración (TTL):** **24 horas**, minimizando la ventana de exposición en caso de extravío del dispositivo.
- **Naturaleza Stateless:** El servidor no requiere almacenar sesiones en memoria RAM ni en base de datos; la validez del token se verifica matemáticamente comprobando la firma criptográfica con la clave `JWT_SECRET`.

NIVEL 4: Control de Acceso Basado en Roles (RBAC)

Se implementa el **Principio de Menor Privilegio (PoLP - Principle of Least Privilege)**:

- **`ADMIN`**: Control total: gestión de catálogo, anulación de pedidos, visualización de métricas de ingresos y configuración de usuarios.
- **`OPERADOR` / `ALMACÉN`**: Actualización de estados logísticos (**Preparación, Camino, Entregado**) y control de existencias físicas.
- **`VENDEDOR`**: Registro de nuevos pedidos, consulta de stock disponible y emisión de comprobantes. No tiene acceso a márgenes financieros ni auditoría.
- **`CLIENTE` (Público)**: Consulta de catálogo y tracking de pedidos únicamente mediante su código alfanumérico unívoco (**LFT-NNN**).

Implementación técnica (`backend/src/middlewares/auth.middleware.ts`):

```
export const requireRole = (allowedRoles: string[]) => {
  return (req: AuthenticatedRequest, res: Response, next: NextFunction) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        error: { code: 'FORBIDDEN', message: 'Permisos insuficientes para esta operación' }
      });
    }
    next();
  };
};
```

NIVEL 5: Validación Estricta de Esquemas e Inmunidad a Payloads Maliciosos (Zod)

Para evitar inyecciones de datos no tipados o manipulación de parámetros (*Mass Assignment Vulnerability*), todas las rutas validan el cuerpo de la petición (**req.body**) contra esquemas tipados con **Zod**:

```
const createOrderSchema = z.object({
  cliente: z.object({
    nombre: z.string().min(2).max(100),
    telefono: z.string().regex(/^51[0-9]{9}$/),
    direccion: z.string().min(5).max(250)
  }),
  items: z.array(z.object({
    productoId: z.string().uuid(),
    cantidad: z.number().int().positive()
  })).min(1),
  metodoPago: z.enum(['Yape', 'Plin', 'Transferencia BCP', 'Contra Entrega'])
});
```

Cualquier campo inyectado adicional (como **precio: 0** o **estado: 'Entregado'**) es descartado inmediatamente con un error HTTP 400 antes de llegar a la lógica de negocio.

NIVEL 6: Prevención Total de Inyección SQL (Patrón Repositorio Parametrizado)

El sistema **no utiliza concatenación de cadenas SQL en ninguna línea de código**. Todas las interacciones con PostgreSQL se realizan mediante consultas preparadas con parámetros tipados:

```
// SEGURO: El motor de base de datos trata el parámetro $1 puramente como dato, nunca como código ejecutable
```

```
const result = await pool.query(  
  'SELECT id, codigo, total, estado FROM orders WHERE codigo = $1 LIMIT 1',  
  [orderId]  
);
```

Incluso si un usuario ingresa payloads maliciosos como ' OR 1=1; DROP TABLE users; --, PostgreSQL busca un pedido cuyo código literal sea idéntico a esa cadena de texto, resultando en 0 coincidencias y protegiendo la base de datos al 100%.

NIVEL 7: Trazabilidad, No Repudio y Auditoría Forense (Ley N° 29733)

Conforme a la normativa peruana de protección de datos personales y buenas prácticas bancarias:

8. Las contraseñas en texto plano nunca se registran en los logs de consola ni en archivos de texto.

9. Cada operación de modificación o eliminación lógica queda registrada con:

* Identificador del usuario que ejecutó la acción.

* Dirección IP de procedencia y cabecera User-Agent.

* Timestamp en formato ISO 8601 UTC.

Estado anterior (before) y estado nuevo (after).*

4. GUÍA DE SUSTENTACIÓN RÁPIDA ANTE EL JURADO (RESPUESTA EN 2 MINUTOS)

Si el profesor o jurado pregunta:

> "Explíqueme la seguridad de su sistema. ¿Cómo garantizan que un atacante no vulnere su aplicación?"

Respuesta Técnica Estructurada (en 4 pasos):

10. Defensa en Profundidad:

"Profesor, no dependemos de un único mecanismo; aplicamos una arquitectura de Defensa en Profundidad con 7 capas concéntricas, desde la red hasta el almacenamiento."

11. Criptografía Formal:

"En autenticación usamos Bcrypt con factor de costo 10 y salting aleatorio de 128 bits, lo que hace inviable cualquier ataque por fuerza bruta o tablas arcoíris. Las sesiones son gestionadas con JWT firmados con HMAC-SHA256 y expiración a las 24 horas, controlando el acceso mediante un middleware RBAC basado en el principio de menor privilegio."

12. Inmunidad contra OWASP Top 10:

"Protegemos las consultas con el patrón Repositorio y sentencias 100% parametrizadas en PostgreSQL, impidiendo la inyección SQL; y en la capa HTTP mitigamos XSS y Clickjacking mediante cabeceras Helmet y validación estricta de esquemas tipados con Zod."

13. Verificación Automatizada:

"Toda esta arquitectura está respaldada por una suite automatizada de pruebas de seguridad en Jest (security.test.ts) y auditorías estáticas con npm audit, registrando cero vulnerabilidades."